

M09 / PROTECTED ANSWERS

M09 Explained Reviews

Standalone Zero-to-Job-Ready Data Engineer 2026

Study, practise, check and explain.

A complete learning route

Start with the lesson video. Read the worked explanation beside it, pause to reproduce the example, then solve the supplied task. The books contain your study notes; keep your own work and results as evidence.

Use the protected answers after saving your first attempt. A changed retry shows what you can now do independently. There is no forced timer during learning.

One course. M00-M19. 264 lessons and 50 project phases.

Contents

Tap a topic or use the PDF bookmarks. These numbers are the actual pages in this edition.

Read this first 3

DE09-01 review - ETL/ELT boundary 3

DE09-02 review - layer contracts 3

DE09-03 review - deterministic transformations 4

DE09-04 review - deduplication and key integrity 4

DE09-05 review - schema checks and contracts 4

DE09-06 review - source-to-target reconciliation 4

DE09-07 review - incremental processing and safe reruns 5

DE09-08 review - failures, retries, quarantine and atomic state 5

DE09-09 review - lineage, run metadata and auditability 5

DE09-10 review - build and defend a reliable batch pipeline 6

DE09-11 review - dbt project anatomy and deterministic dependency graph 6

DE09-12 review - executable tests, severity and CI boundary 6

DE09-13 review - incremental models, materialization, documentation and handoff 6

Read this first

Open this book after saving your own attempt. Compare the reasoning and result, repair the cause of a mistake, then close the answer and try changed inputs.

OPEN AFTER A SAVED GENUINE ATTEMPT

Standalone Zero-to-Job-Ready Data Engineer 2026

Open only after you saved a genuine attempt. Each review repairs the matching lesson; it is not a substitute for the first attempt. Compare expected evidence, identify the smallest wrong assumption, close this book, then solve the lesson's changed retry from a clean state.

DE09-01 review - ETL/ELT boundary

A correct design can be ETL-like or ELT-like. Start by marking the first durable copy of source evidence. In the Python route, the source batch is preserved before transformation; in the dbt route, `setup_raw_duckdb.py` loads source versions into `raw.raw_orders` before dbt models transform them. In both cases, state must advance only after the accepted target effect is durable and validated.

Worked repair: if your diagram says “API -> transform -> serving” and has no durable raw/landing copy, ask what happens when tomorrow's transformation code is found to be wrong but the API no longer retains yesterday's response. The missing replay source is the defect. Add durable raw evidence, then place validation/quarantine and serving downstream. If your ELT answer says “load everything, therefore quality is solved,” the defect is different: raw retention and downstream contract enforcement are separate responsibilities.

Validate: identify raw evidence, transform location, quarantine/rejection route, serving boundary and state-commit point. A fashionable acronym is not evidence. Changed retry: redesign for a source that retains only seven days and charges for rereads.

DE09-02 review - layer contracts

Use O6003 to walk the layers. Raw preserves the region exactly as received: east . Staging normalizes it to East. Serving calculates $1 * 12000 * (1 - 0.05) = 11400.00$ and carries lineage. The raw layer may legitimately contain malformed or awkward values because its job is evidence; serving may not silently expose unresolved values.

Worked repair: if the serving total is wrong, do not jump directly to the dashboard formula. Trace serving -> intermediate/dedup -> staging -> raw and stop at the first layer where the key/value/count diverges. If you copied the cleansing rule into both staging and serving, choose one owner and remove the duplicated rule. If a new `sales_channel` field arrives, raw can preserve it immediately while staging/serving wait for an approved contract.

Validate: one row/grain statement per layer, one owner for each rule, and a reproducible path from raw to serving.

DE09-03 review - deterministic transformations

The supplied controls include O6001 = 1530.00 and O6005 = 6300.00. Reordering source rows must not change accepted business keys or values. Determinism means explicit inputs, types, grain, stable tie handling and controlled money rounding; it does not mean every physical output byte must be identical.

A dangerous query is `row_number()` over (partition by `order_id` order by `updated_at` desc) when

`updated_at` can tie. That query runs, but the winner is not justified. If the source has no trustworthy secondary key, the safe answer is to surface/quarantine ambiguity rather than invent file-order precedence. The repaired dbt project now does exactly this: `int_order_conflicts` exposes ambiguous latest timestamps and `int_orders_deduped` keeps only unambiguous latest candidates.

Validate by shuffling input order, rerunning, comparing key/value parity and proving a tied latest conflict does not silently become a winner.

DE09-04 review - deduplication and key integrity

For the duplicate fixture, the accounting control is 4 source = 1 staged + 2 quarantined + 1 older discarded. O6101 at 09:00 with qty 2 is the safe latest version. O6102 has two conflicting latest candidates at the same timestamp, so both are ambiguous and must not be guessed. Serving therefore contains one accepted row worth 1700.00.

Worked repair: `DISTINCT` is wrong because the conflicting O6102 rows are not identical. `drop_duplicates(keep='last')` is also wrong because input order becomes business logic. A timestamp-only `row_number()` has the same problem. First group by business key, find the latest timestamp, then ask how many candidates exist at that latest timestamp. `Count=1` is safe; `count>1` needs a trusted source-owned tie-break or quarantine/conflict handling.

Validate: every source row has a disposition and exactly one accepted latest row exists per safe business key.

DE09-05 review - schema checks and contracts

`batch_schema_drift.csv` renames `region` to `sales_region` and omits `discount_pct`. Expected status is `FAILED_CONTRACT`. Preserve the raw file and observed header, compare it with the expected contract, and leave processed state unchanged.

Do not “repair” by silently inventing `discount_pct = 0`; that changes business meaning. Do not rename unknown columns automatically just because the names look similar. Decide whether the producer changed the contract, whether the consumer supports a new version, and who approved the mapping.

Validation checklist: raw hash retained, observed vs expected schema recorded, target not partially updated, state unchanged, explicit compatibility decision documented.

DE09-06 review - source-to-target reconciliation

Batch 001 controls: 6 source rows -> 6 serving rows, total 22930.00. After batch 002 the target is 9 rows / 84870.00. The net increase is 61940.00: O6002 changes by +450, O6007 adds 52250, O6008 adds 3240 and O6009 adds 6000.

Worked repair: if target row count is right but money is wrong, row reconciliation alone is insufficient. Compare changed/new keys and money movement. If money is right but duplicate keys exist, the grand total still does not prove correct grain. Reconcile independently by source rows/dispositions, accepted keys, target keys and amount deltas.

A tolerance must have a business reason (for example approved rounding), not “the numbers are close.” Save the equation you used so another engineer can reproduce the proof.

DE09-07 review - incremental processing and safe reruns

After batch 002 has already been applied, a normal replay must not multiply the target. The intentional replay stages the same versions but applies zero target changes; serving remains 9 rows / 84870.00.

Separate three ideas: source-file identity can prevent processing an identical file twice; a watermark can bound which source versions are fetched; a target-side idempotent/upsert rule prevents duplicate effects. None of these alone solves arbitrary late corrections outside the chosen boundary.

Recovery: compare pre-run target/state, run the same input again, then compare post-run key set, total and state. If any target value changes without a newer source version, locate the non-idempotent side effect before advancing state.

DE09-08 review - failures, retries, quarantine and atomic state

Bad-batch control: 5 source, 1 staged, 4 quarantined, quarantine ratio 0.80. That breaches the quality threshold, so target/state must not advance. A simulated database lock is transient: attempt 1 can fail and bounded attempt 2 can succeed. A deliberately injected failure after staging must leave target/state unchanged.

The repair question is “what class of failure is this?” Invalid facts should be quarantined/rejected, not retried forever. A transient lock may be retried with a bound. A programming/schema defect should fail loudly and await code/contract repair. Atomic state means the externally visible success marker cannot move ahead of

the durable target effect.

Validate with before/after target count+total+state and retained failure evidence, not a green log line.

DE09-09 review - lineage, run metadata and auditability

Trace O6007 backward: serving row -> source_run_id -> run manifest -> batch_002_incremental.csv plus source SHA -> raw record -> deterministic calculation 52250.00. That is operational lineage: it explains which actual run/input produced a result.

dbt graph lineage answers a related but different question: which declared models depend on which sources/models. A DAG edge cannot replace run metadata, source hash, code version or observed counts.

Worked repair: if you cannot answer “which exact source bytes and run produced this serving row?”, add the missing run/source identifiers. If you cannot answer “which transformation node depends on this model?”, repair project graph/docs. Keep both forms of evidence.

DE09-10 review - build and defend a reliable batch pipeline

A defensible sequence is: receive/hash -> raw -> contract -> normalize -> dedup/conflict -> reconcile -> quality gate -> transactional serving -> target controls -> atomic state -> final manifest. The fresh retest fixture in a clean runtime should yield 3 rows / 61980.00.

Do not defend the pipeline by reading its file names. Demonstrate one happy path and one failure path. Show where raw evidence survives, where bad rows go, why a rerun is safe, how money/keys reconcile and why state cannot lie after a partial failure. Performance tuning comes after these correctness boundaries are proven.

If a checkpoint is weak, repair the smallest missing control and rerun the changed case rather than rewriting the whole pipeline.

DE09-11 review - dbt project anatomy and deterministic dependency graph

The logical graph is now: raw.raw_orders -> stg_orders -> int_order_conflicts + int_orders_deduped -> fct_orders / fct_orders_incremental. source() declares an external/raw

relation; ref() declares a dependency on a dbt-managed node.

The important repair is tie safety. int_order_conflicts finds order IDs with more than one candidate at the latest timestamp. int_orders_deduped retains only keys whose latest timestamp has exactly one candidate. The singular test assert_no_ambiguous_latest_orders.sql returns conflict rows; therefore a clean build/test must fail when ambiguity exists instead of silently choosing a winner.

Learner-PC proof: run python setup_raw_duckdb.py, dbt debug --profiles-dir ., then dbt build

--profiles-dir .. Insert a changed tied-latest conflict and expect the conflict test to fail. Repair the source/tie contract; do not add arbitrary file order.

DE09-12 review - executable tests, severity and CI boundary

Structural tests (not_null, unique, accepted_values) and singular SQL tests protect different promises. The negative-net-sales singular test passes only when it returns zero rows. The new ambiguous-latest singular test passes only when int_order_conflicts is empty.

To learn this, deliberately create one controlled violation and predict the exact failing test before running dbt build --profiles-dir .. Save the failure. Repair the cause or deliberately update an approved contract, then rerun. Deleting/weakening the test only to make CI green is not a repair.

Severity is a risk decision. A warning is appropriate only when downstream use can continue safely and ownership/remediation are explicit. A blocking contract should fail the build.

DE09-13 review - incremental models, materialization, documentation and handoff

The supplied incremental model uses `>= max(updated_at)` with `unique_key='order_id'`. The inclusive boundary intentionally rereads max-timestamp rows so a new key tied at the current maximum is not skipped. It does not detect arbitrary older late corrections outside the predicate.

Worked proof: add a new order at exactly the target maximum timestamp; the inclusive predicate should consider it. Then create an older correction and predict it is missed. That is the point where you choose an explicit lookback, CDC, periodic full refresh or reconciliation strategy based on the source contract.

Generate/inspect docs on the learner PC with `dbt docs generate --profiles-dir ..`. A useful handoff records materializations, unique keys, incremental predicate, full-refresh/rebuild route, tests/conflict policy, known late-data limitation and reproducible setup. The receiver should not have to guess what state means.